

可以。从[代码实现](#)看，OPD 和传统蒸馏最大的区别其实不是 KL 公式，而是 [prefix / trajectory 到底是谁生成的](#)。传统蒸馏通常是：

```
# 固定 teacher 数据 / ground-truth 数据
for batch in dataloader:
    input_ids = batch["input_ids"]

    with torch.no_grad():
        teacher_logits = teacher(input_ids).logits

    student_logits = student(input_ids).logits

    loss = kl_div(
        student_logits,
        teacher_logits,
    )

    loss.backward()
    optimizer.step()
```

这里 teacher 和 student 看到的是[同一份固定 sequence](#)：

```
Prompt + Teacher/GT completion
```

也就是：

```
x, y1, y2, y3, y4
```

这些 prefix 来自静态数据集，而不是当前 Student。所以数学上是：

$$(x, y) \sim D$$

然后：

$$L_{\text{KD}} = \sum_t D_{\text{KL}}(p_T(\cdot|x, y_{<t}) || p_S(\cdot|x, y_{<t}))$$

这正是论文所说的 classical token-level KD：它使用静态 prefix，因此训练时默认的是“正确历史”，而部署时 Student 会访问自己的状态。

1. OPD 代码最大的变化：先 student.generate()

OPD 变成：

```
for prompts in dataloader:

    # ① Student 自己生成 trajectory
    with torch.no_grad():
        generated_ids = student.generate(
            prompts,
            do_sample=True,
            temperature=1.0,
        )

    # ② Student 在自己生成的 prefix 上重新 forward
    student_logits = student(
        generated_ids
    ).logits

    # ③ Teacher 也看 Student trajectory
    with torch.no_grad():
        teacher_logits = teacher(
            generated_ids
        ).logits

    # ④ 做 distillation
    loss = kl_div(
        student_logits,
        teacher_logits,
    )

    loss.backward()
    optimizer.step()
```

也就是多出了最关键的一句：

```
generated_ids = student.generate(...)
```

所以训练数据变成：

$$y \sim p_{\theta}(\cdot|x)$$

而不是：

$$y \sim D$$

论文对 On-Policy 的正式定义就是：

Student 的训练数据在训练时由 Student 当前 policy p_{θ} 自己采样，因此随着 θ 更新，数据分布也不断变化。

2. 举一个具体 token 例子

假设 prompt：

```
17 * 23 =
```

Teacher 正确轨迹：

```
17 * 23 = 391
```

Student 当前却会生成：

```
17 * 23 = 381
```

DataLoader 里面已经保存:

```
input_ids = tokenize(
    "17 * 23 = 391"
)
```

forward:

```
teacher_logits = teacher(
    "17 * 23 = 391"
)

student_logits = student(
    "17 * 23 = 391"
)
```

所以训练的 prefix 是:

```
17
17 *
17 * 23
17 * 23 =
17 * 23 = 3
17 * 23 = 39
```

Student 永远不会在训练中看到:

```
17 * 23 = 38
```

这个自己真正会进入的错误 state。

首先:

```
rollout = student.generate(  
    "17 * 23 ="  
)
```

得到:

```
381
```

然后:

```
trajectory = "17 * 23 = 381"  
  
teacher_logits = teacher(trajectory)  
student_logits = student(trajectory)
```

于是 Teacher 会在这些 prefix 上提供监督:

```
17 * 23 =  
17 * 23 = 3  
17 * 23 = 38    ← 关键  
17 * 23 = 381
```

也就是 Teacher 真正被问:

```
teacher(  
    "17 * 23 = 38"  
)
```

它可能得到:

```
{
  "1": 0.001,
  "9": 0.95,
  "0": 0.01,
  ...
}
```

而 Student:

```
{
  "1": 0.70,
  "9": 0.15,
  "0": 0.05,
  ...
}
```

于是 KL 会纠正:

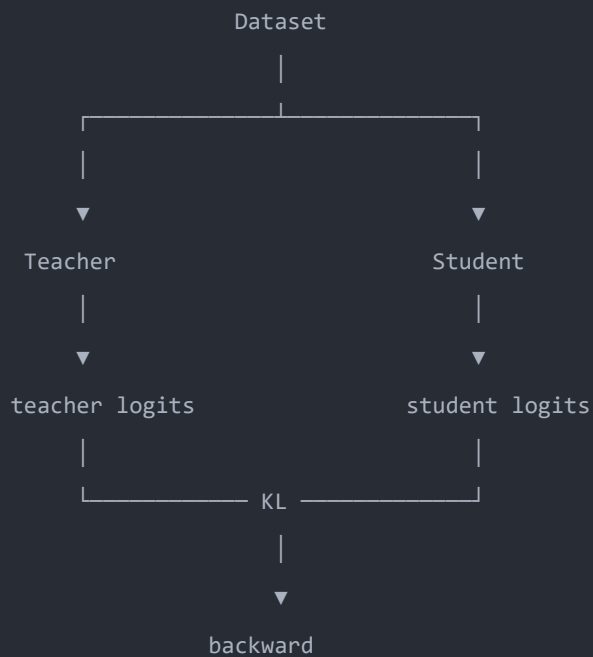
```
Student 实际犯错的位置
    ↓
teacher(student_prefix)
    ↓
dense token supervision
```

这就是 OPD 的本质。

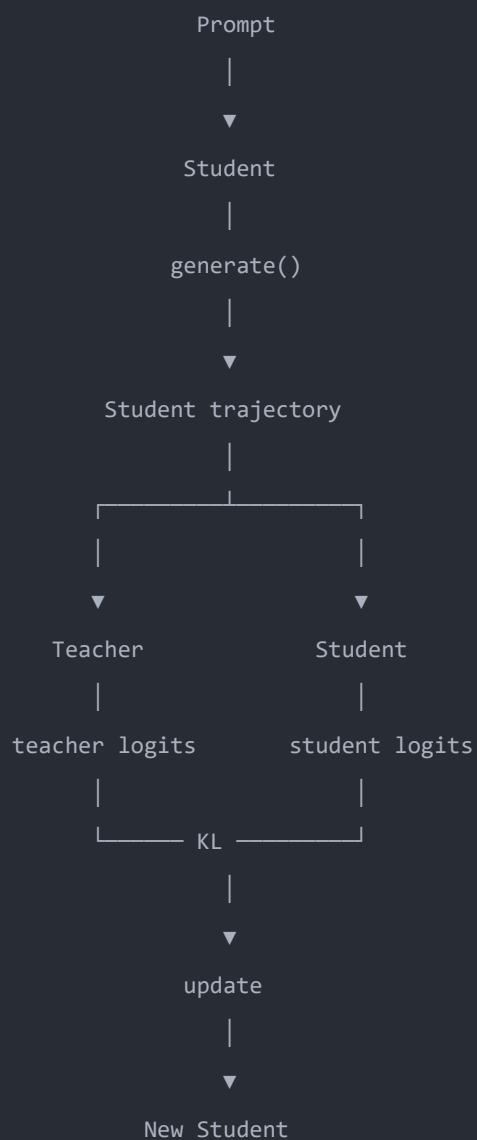
3. 所以两者的数据流完全不同

传统 KD:





数据集是中心。而 OPD:



```
|  
└─ generate again
```

Student policy 是整个 pipeline 的中心。

4. 从 PyTorch 代码看，区别可以缩成这一行

传统 KD:

```
tokens = dataset_tokens
```

OPD:

```
tokens = student.generate(prompts)
```

但别被这一行骗了。这一行会导致整个训练系统完全不同。因为:

```
student.generate()
```

是 autoregressive decode。所以 OPD 从:

```
forward + backward
```

变成:

```
forward + backward
```

```
decode
+
teacher forward
+
student forward
+
backward
```

论文第 6 章正是把 OPD 的主要计算拆成 Student rollout、Teacher scoring 和 Student update 三阶段。

5. 一个更接近真实训练的传统 KD 实现

假设我们蒸馏 Qwen-32B → Qwen-7B。



```

import torch
import torch.nn.functional as F

def forward_kl(student_logits, teacher_logits):

    teacher_prob = F.softmax(
        teacher_logits.float(),
        dim=-1
    )

    student_logprob = F.log_softmax(
        student_logits.float(),
        dim=-1
    )

    teacher_logprob = F.log_softmax(
        teacher_logits.float(),
        dim=-1
    )

    return (
        teacher_prob *
        (teacher_logprob - student_logprob)
    ).sum(-1).mean()

```

训练:

```

for batch in dataloader:

    tokens = batch["input_ids"]

    # -----
    # Teacher
    # -----

    with torch.no_grad():

        teacher_logits = teacher(
            input_ids=tokens
        ).logits[:, :-1]

```

```

# -----
# Student
# -----

student_logits = student(
    input_ids=tokens
).logits[:, :-1]

# -----
# KD
# -----

loss = forward_kl(
    student_logits,
    teacher_logits
)

optimizer.zero_grad()

loss.backward()

optimizer.step()

```

这个过程中：



```
tokens
```

整个 epoch 都是固定数据。 所以：



```

Step 1: dataset batch #1
Step 2: dataset batch #2
Step 3: dataset batch #3
...

```

Student 参数变化，并不会改变训练 sequence。

6. OPD 版本

现在改成 OPD:

```
for prompt_batch in dataloader:

    prompt_ids = prompt_batch["input_ids"]

    # =====
    # Stage 1
    # Student rollout
    # =====

    with torch.no_grad():

        rollout_ids = student.generate(
            input_ids=prompt_ids,

            max_new_tokens=1024,

            do_sample=True,

            temperature=1.0
        )

    # =====
    # Stage 2
    # Teacher scoring
    # =====

    with torch.no_grad():

        teacher_logits = teacher(
            input_ids=rollout_ids
        ).logits[:, :-1]

    # =====
    # Stage 3
    # Student scoring
    # =====
```

```

student_logits = student(
    input_ids=rollout_ids
).logits[:, :-1]

# =====
# KL
# =====

loss = forward_kl(
    student_logits,
    teacher_logits
)

optimizer.zero_grad()

loss.backward()

optimizer.step()

```

第 N 步:

```

rollout_ids = student_N.generate()

```

第 N+1 步:

```

rollout_ids = student_N_plus_1.generate()

```

由于:

```

student_N != student_N_plus_1

```

所以:

```


```

```
rollout_N != rollout_N_plus_1
```

因此训练 distribution 本身也在变化：

$$p_{\theta_0} \rightarrow p_{\theta_1} \rightarrow p_{\theta_2} \rightarrow \dots$$

这就是论文所说的 **non-stationary optimization landscape**。

7. 还有一个重要区别：OPD 通常只算 completion

真实实现不会：

```
loss = kl(all_tokens)
```

因为 prompt 不需要学习。 假设：

```
[P P P P][R R R R R R]
prompt   rollout
```

创建 mask：

```
loss_mask = torch.zeros_like(rollout_ids)

loss_mask[:, prompt_len:] = 1
```

于是：

```
token_loss = kl_per_token(
    student_logits,
    teacher_logits
)

loss = (
```

```
token_loss * loss_mask[:, 1:]
).sum() / loss_mask[:, 1:].sum()
```

最终只有 Student 自己生成的：

```
response / reasoning trajectory
```

参与 OPD。

8. Reverse KL 版本更能体现 OPD

前面是：

$$D_{KL}(p_T \| p_S)$$

也就是 Forward KL。很多 reasoning OPD 更关心 Reverse KL：

$$D_{KL}(p_S \| p_T)$$

代码：

```
def reverse_kl(student_logits, teacher_logits):

    student_prob = F.softmax(
        student_logits.float(),
        dim=-1
    )

    student_logprob = F.log_softmax(
        student_logits.float(),
        dim=-1
    )

    teacher_logprob = F.log_softmax(
        teacher_logits.float(),
        dim=-1
    )
```

```

kl = (
    student_prob *
    (
        student_logprob -
        teacher_logprob
    )
).sum(-1)

return kl

```

然后：

```

token_loss = reverse_kl(
    student_logits,
    teacher_logits
)

loss = (
    token_loss * completion_mask
).sum() / completion_mask.sum()

```

这里的关键仍然不是 RKL。即使你传统 KD 用 RKL：

```

tokens = teacher_dataset
loss = reverse_kl(...)

```

它仍然是 **off-policy distillation**。真正决定是不是 OPD 的，是：

```

tokens = student.generate()

```

而不是：

```

loss = reverse_kl()

```

这个区别特别值得记住。

9. 所以不要把 OPD 和 KL 方向混在一起

实际上有四种组合：

trajectory	loss	类型
Teacher/Data	FKL	普通 KD
Teacher/Data	RKL	off-policy RKL KD
Student	FKL	OPD
Student	RKL	OPD

论文的 unified framework 正是在强调：

sampling policy

和

divergence

是两个不同的维度：

$$L_{\text{OPD}} = E_{y \sim \pi_{\text{mix}}} \sum_t D_f(p_T, p_S)$$

其中：



mix

決定：

数据谁生成。而：

Df

决定：

怎么匹配 Teacher。

10. GKD 的代码就更清楚了

GKD 并不是一定 100% Student rollout, 而是：

$$\pi_{\text{mix}} = \lambda p_S + (1 - \lambda) p_{\text{data}}$$

代码相当于：

```
if torch.rand(1) < lambda_on_policy:

    # on-policy
    tokens = student.generate(
        prompt_ids
    )

else:

    # off-policy
    tokens = dataset_completion
```

然后统一：

```
teacher_logits = teacher(tokens)

student_logits = student(tokens)

loss = divergence(
    teacher_logits,
    student_logits
)
```

所以：

```
lambda = 0
```

就是传统 KD。

```
lambda = 1
```

就是纯 OPD。论文也是这样描述 GKD 的，并指出其混合策略用 λ 连续连接 off-policy 和 on-policy。

11. OPD 为什么很像 GRPO?

如果从 training engine 角度看，二者已经很接近。GRPO：

```
# 1 rollout
responses = actor.generate(prompts)

# 2 reward
rewards = verifier(responses)

# 3 old log probs
old_log_probs = actor.compute_log_probs(
    responses
)

# 4 advantage
advantages = compute_group_advantage(
    rewards
)

# 5 policy update
loss = grpo_loss(
    log_probs,
    old_log_probs,
```

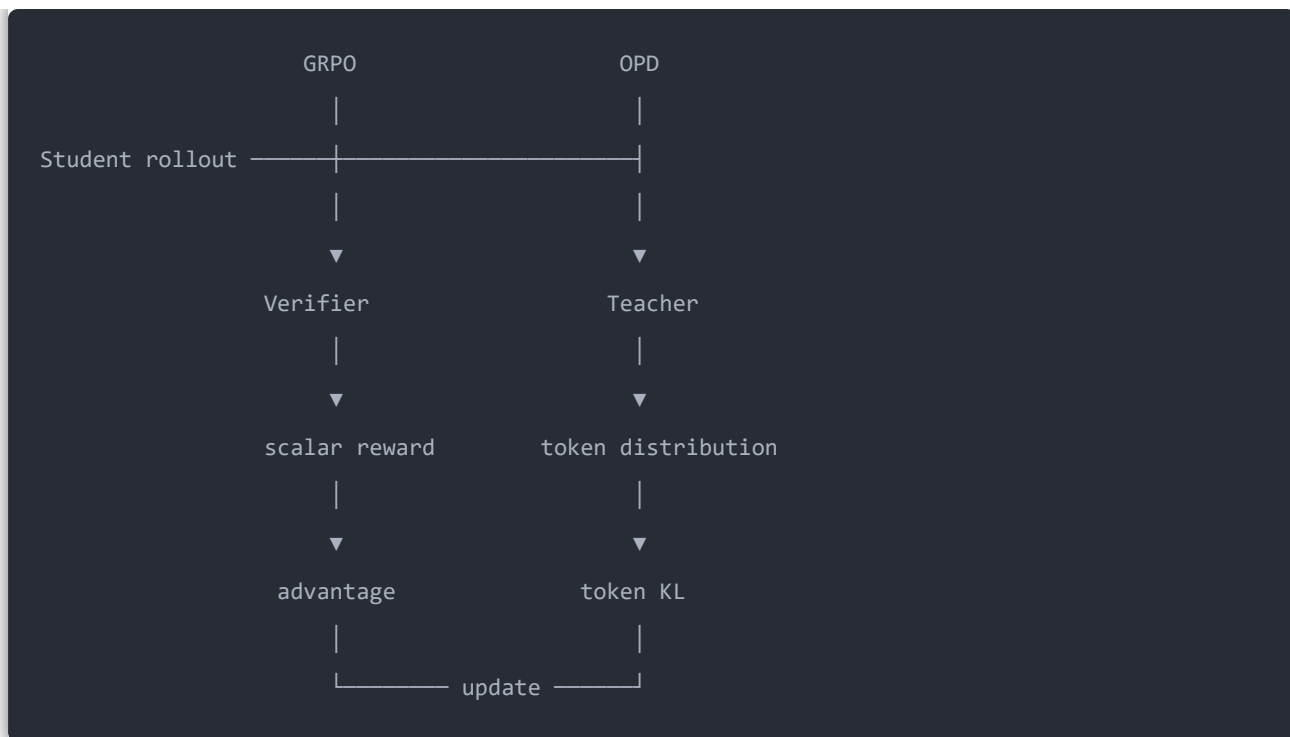
```
    advantages  
)
```

OPD:

```
● ● ●  
  
# 1 rollout  
responses = student.generate(prompts)  
  
# 2 teacher score  
teacher_logits = teacher(responses)  
  
# 3 student score  
student_logits = student(responses)  
  
# 4 dense teacher signal  
loss = reverse_kl(  
    student_logits,  
    teacher_logits  
)  
  
# 5 update  
loss.backward()
```

并排:





所以可以非常粗略地说：

$$GRPO = Student\ rollout + Sparse\ reward$$

而：

$$OPD = Student\ rollout + Dense\ teacher\ distribution$$

这也是论文为什么专门讨论 OPD 与 KL-constrained RL 的关系；它指出两类方法共享 rollout-score-update 的计算结构，区别主要在监督信号来源。

12. 从 SGLang/vLLM 系统角度，差别更明显

传统蒸馏的 pipeline：

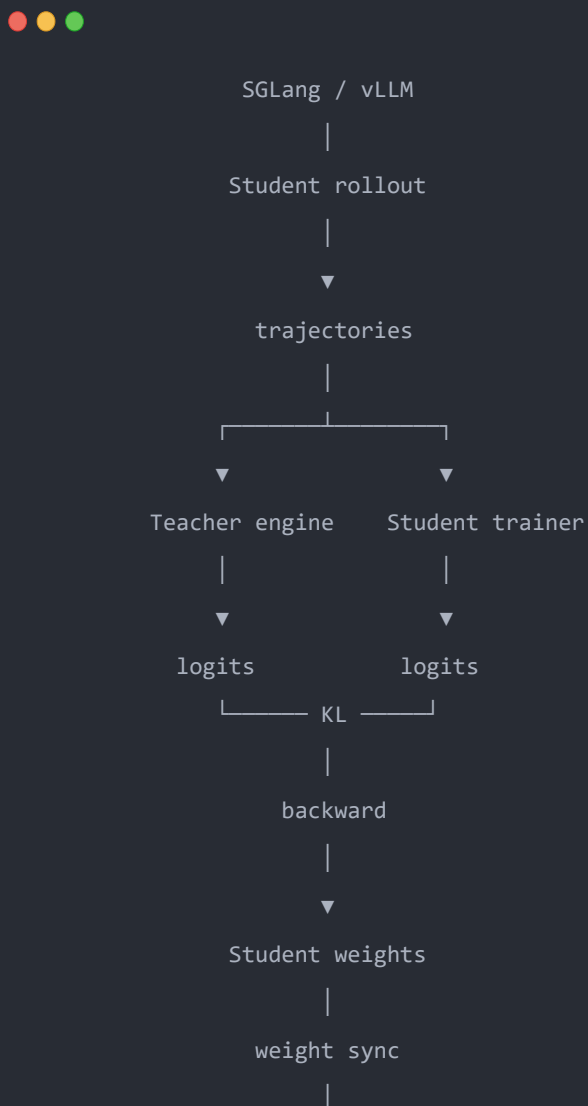




甚至 Teacher 都不一定在线。可以提前：

```
torch.save(teacher_logits)
```

之后只训练 Student。但 OPD：



rollout engine

所以 OPD 会碰到 RL 系统里同样的问题：

- rollout engine;
- training engine;
- weight synchronization;
- stale rollout;
- continuous batching;
- generation/training disaggregation;
- Teacher inference;
- logit communication;
- async pipeline。这也是为什么 OPD 在系统层面比传统 KD 贵得多。

13. 真正工业级 OPD 的伪代码

如果你以后自己用 SGLang 做，可以理解成：

```
student_rollout_engine = SGLangServer(  
    model="student"  
)  
  
teacher_engine = SGLangServer(  
    model="teacher",  
    return_logits=True  
)  
  
student_trainer = TrainingEngine(  
    model="student"  
)
```

训练 loop：

```

for step in range(num_steps):

    # =====
    # 1. rollout
    # =====

    trajectories = student_rollout_engine.generate(
        prompts
    )

    # =====
    # 2. teacher scoring
    # =====

    teacher_logits = teacher_engine.forward(
        trajectories
    )

    # =====
    # 3. student training
    # =====

    student_logits = student_trainer.forward(
        trajectories
    )

    loss = reverse_kl(
        student_logits,
        teacher_logits
    )

    student_trainer.backward(loss)

    student_trainer.step()

    # =====
    # 4. sync rollout weights
    # =====

    if step % sync_interval == 0:

```

```
student_rollout_engine.update_weights(  
    student_trainer.weights()  
)
```

你会发现，这已经非常像：



OpenRLHF / VeRL / GRPO

的架构了。唯一主要变化：



Reward Model / Verifier



Teacher logits

最后用代码记住 OPD 和普通蒸馏

普通蒸馏：



```
# 谁生成训练 trajectory?  
trajectory = dataset_or_teacher_data  
  
student_logits = student(trajectory)  
  
teacher_logits = teacher(trajectory)  
  
loss = KL(student_logits, teacher_logits)
```

OPD：



```
# 谁生成训练 trajectory?  
trajectory = student.generate(prompt)  
  
student_logits = student(trajectory)
```

```
teacher_logits = teacher(trajecory)

loss = KL(student_logits, teacher_logits)
```

所以真正的分界线就是：

```
trajectory = ?
```

而不是：

```
loss = ?
```

`trajectory = student.generate()` 才是 On-Policy Distillation 的灵魂。如果再和你熟悉的 GRPO 对齐来看，最有价值的下一步其实是把三份训练代码并排拆：

```
SFT
VS
传统 KD
VS
OPD
VS
GRPO
```

一直拆到 `input_ids / rollout / logits / log_probs / reward / advantage / loss.backward()` 每一步。我可以直接给你写一份接近 HuggingFace + SGLang/verl 实现的四套完整伪代码，这样你从代码上一眼就能看出它们到底差在哪里。